

AUTOM-06: Dynatrace SDKs

Series: AUTOM — Dynatrace Automation | **Notebook:** 6 of 9 | **Created:** January 2026 | **Last Updated:** 09/18/2026

Dynatrace publishes official TypeScript SDK clients (`@dynatrace-sdk/*`) for programmatic access to the platform, auto-generated from OpenAPI specifications. There is no official Python SDK package — Python automation calls the REST APIs directly.

Table of Contents

- 1. [Introduction](#)
- 2. [TypeScript SDK](#)
- 3. [Python: Calling the REST APIs](#)
- 4. [Common Patterns](#)
- 5. [MCP Server Integration](#)
- 6. [Next Steps](#)

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Node.js 18+	For TypeScript SDK
Python 3.9+ with <code>requests</code>	For the Python REST examples
API Token	Token with required scopes
Development Environment	VS Code or similar IDE

Learning Objectives

By the end of this notebook, you will:

- Understand the Dynatrace SDK architecture
 - Know how to use the TypeScript SDK clients and call the REST APIs from Python
 - Be able to query data and manage configurations programmatically
 - Build custom automation applications
-

1. Introduction

Available SDKs

SDK	Language	Use Case
@dynatrace-sdk/*	TypeScript/JavaScript	Dynatrace apps, app functions, workflow JavaScript tasks
(none)	Python	No official Python SDK is published (dt-sdk is not on PyPI). Call the REST APIs with requests or a clearly labelled community library

SDK vs Direct API

Aspect	SDK	Direct API
Type safety	Yes	No
Auto-completion	Yes	No
Authentication	Built-in	Manual
Pagination	Handled	Manual
Error handling	Structured	Raw HTTP

Client Libraries

Client	Purpose
QueryClient	Execute DQL queries
SettingsClient	Manage Settings 2.0 objects
EntitiesClient	Query entity topology
MetricsClient	Query and ingest metrics
EventsClient	Query and ingest events
LogsClient	Query and ingest logs

2. TypeScript SDK

Installation

```
npm install @dynatrace-sdk/client-query
npm install @dynatrace-sdk/client-classic-environment-v2
```

(`@dynatrace-sdk/client-core` and `@dynatrace-sdk/client-settings-v2` are not published on npm; Settings 2.0 objects are reached through `client-classic-environment-v2` .)

Configuration

The `@dynatrace-sdk/*` clients are built for code that runs on the Dynatrace platform — apps, app functions and workflow JavaScript tasks — where the platform runtime supplies authentication. There is no `setEnvConfig` step to call. For scripts that run outside the platform (CI jobs, local tooling), call the REST APIs directly as in §3.

Query Example

```
import { queryClient } from '@dynatrace-sdk/client-query';

async function queryHosts() {
  const result = await queryClient.query({
    body: {
      query: `
        fetch dt.entity.host
        | fieldsAdd name = entity.name, osType
        | sort name asc
        | limit 10
      `,
      defaultTimeframeStart: 'now-24h',
      defaultTimeframeEnd: 'now'
    }
  });

  console.log('Hosts:', result.result?.records);
  return result.result?.records;
}
```

Settings Management

```
import { settingsObjectsClient } from '@dynatrace-sdk/client-classic-environment-v2';

// List management zones
async function listManagementZones() {
  const response = await settingsObjectsClient.getSettingsObjects({
    schemaIds: 'builtin:management-zones',
    pageSize: 100
  });

  return response.items;
}

// Create management zone
async function createManagementZone(name: string) {
  const response = await settingsObjectsClient.postSettingsObjects({
    body: [{
      schemaId: 'builtin:management-zones',
      scope: 'environment',
      value: {
        name: name,
        rules: []
      }
    }]
  });

  return response;
}
```

Entity Queries

```
import { entitiesClient } from '@dynatrace-sdk/client-classic-environment-v2';

async function getServices() {
  const response = await entitiesClient.getEntities({
    entitySelector: 'type(SERVICE)',
    fields: '+properties,+tags',
    pageSize: 100
  });

  return response.entities;
}
```

3. Python: Calling the REST APIs

Dynatrace does not publish a Python SDK: `pip install dt-sdk` fails because no such package exists on PyPI. Python automation calls the documented REST APIs directly. The examples below use the Settings 2.0 and Entities endpoints of the Environment API v2 with an access token (`Api-Token` header).

Setup

```
pip install requests
```

```
import os
import requests

BASE = os.environ['DT_TENANT_URL'].rstrip('/') #
https://{tenant}.live.dynatrace.com
session = requests.Session()
session.headers['Authorization'] = f"Api-Token {os.environ['DT_API_TOKEN']}"
```

Settings Management

```
def list_settings_objects(schema_id: str) -> list[dict]:
    """All objects of one Settings 2.0 schema, following nextPageKey."""
    items: list[dict] = []
    params = {'schemaIds': schema_id, 'pageSize': 100}
    while True:
        r = session.get(f'{BASE}/api/v2/settings/objects', params=params,
            timeout=30)
        r.raise_for_status()
        body = r.json()
        items.extend(body.get('items', []))
        if not body.get('nextPageKey'):
            return items
        # With nextPageKey set, the API requires all other query parameters to
        # be omitted
        params = {'nextPageKey': body['nextPageKey']}

def create_settings_object(schema_id: str, value: dict, scope: str =
    'environment') -> list[dict]:
    r = session.post(f'{BASE}/api/v2/settings/objects',
        json=[{'schemaId': schema_id, 'scope': scope, 'value':
            value}], timeout=30)
    r.raise_for_status()
    return r.json()
```

Pick a schema that survives the upgrade to Latest Dynatrace for new automation (e.g. `builtin:davis.anomaly-detectors`); management zones and alerting profiles are on Dynatrace's removed-schemas list (AUTOM-02).

Pagination Handling

```
def get_all_hosts() -> list[dict]:
    """Iterate through all pages of hosts."""
    hosts: list[dict] = []
    params = {'entitySelector': 'type(HOST)', 'pageSize': 500}
    while True:
        r = session.get(f'{BASE}/api/v2/entities', params=params, timeout=30)
        r.raise_for_status()
        body = r.json()
        hosts.extend(body.get('entities', []))
        if not body.get('nextPageKey'):
            return hosts
        params = {'nextPageKey': body['nextPageKey']}
```

4. Common Patterns

Bulk Configuration Export

```
// TypeScript: Export all settings for backup
import { settingsObjectsClient, settingsSchemasClient } from '@dynatrace-
sdk/client-classic-environment-v2';
import * as fs from 'fs';

async function exportAllSettings() {
  // Get all available schemas
  const schemas = await settingsSchemasClient.getAvailableSchemaDefinitions();

  const export_data: Record<string, any[]> = {};

  for (const schema of schemas.items || []) {
    const objects = await settingsObjectsClient.getSettingsObjects({
      schemaIds: schema.schemaId,
      pageSize: 500
    });

    if (objects.items && objects.items.length > 0) {
      export_data[schema.schemaId] = objects.items;
    }
  }

  fs.writeFileSync('settings-export.json', JSON.stringify(export_data, null,
2));
  return export_data;
}
```

Error Handling

```
import { queryClient } from '@dynatrace-sdk/client-query';

async function safeQuery(query: string) {
  try {
    const result = await queryClient.query({
      body: { query }
    });
    return { success: true, data: result };
  } catch (error) {
    // Log and surface the failure; re-throw anything you cannot handle
    console.error('Query failed:', error instanceof Error ? error.message :
error);
    return { success: false, error: String(error) };
  }
}
```

Rate Limiting

```
import time
from functools import wraps

def rate_limited(max_per_second: float):
    """Decorator to rate limit API calls."""
    min_interval = 1.0 / max_per_second
    last_called = [0.0]

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            elapsed = time.time() - last_called[0]
            if elapsed < min_interval:
                time.sleep(min_interval - elapsed)
            result = func(*args, **kwargs)
            last_called[0] = time.time()
            return result
        return wrapper
    return decorator

@rate_limited(10) # Max 10 requests per second
def api_call(entity_id: str) -> dict:
    r = session.get(f'{BASE}/api/v2/entities/{entity_id}', timeout=30)
    r.raise_for_status()
    return r.json()
```

Async Operations

```
// TypeScript: Concurrent queries with rate limiting
import pLimit from 'p-limit';

const limit = pLimit(5); // Max 5 concurrent requests

async function queryMultipleHosts(hostIds: string[]) {
  const queries = hostIds.map(hostId =>
    limit(() => queryClient.query({
      body: {
        query: `
          fetch dt.entity.host
          | filter id == "${hostId}"
        `
      }
    }
  ));
  return Promise.all(queries);
}
```

Reporting Script

```
import pandas as pd

def generate_host_report() -> pd.DataFrame:
    """Generate a host inventory report from the Entities API (uses
    session/BASE from §3)."""
    rows = []
    params = {'entitySelector': 'type(HOST)', 'fields': '+properties',
'pageSize': 500}
    while True:
        r = session.get(f'{BASE}/api/v2/entities', params=params, timeout=30)
        r.raise_for_status()
        body = r.json()
        for e in body.get('entities', []):
            props = e.get('properties', {})
            rows.append({
                'name': e.get('displayName'),
                'osType': props.get('osType'),
                'cpuCores': props.get('cpuCores'),
                'physicalMemory': props.get('physicalMemory'),
            })
        if not body.get('nextPageKey'):
            break
        params = {'nextPageKey': body['nextPageKey']}

    df = pd.DataFrame(rows).sort_values('name')
    df.to_csv('host_inventory.csv', index=False)
    return df
```

Best Practices

Practice	Description
Environment variables	Never hardcode credentials
Type safety	Use the SDK's TypeScript types, or Python type hints on your REST wrappers
Error handling	Catch and handle API errors

Practice	Description
Pagination	Always handle paginated responses
Rate limiting	Respect API limits
Logging	Add structured logging

5. MCP Server Integration

Dynatrace MCP Server

The [Dynatrace MCP Server](#) provides an alternative to traditional SDKs for AI-assisted workflows. It implements the Model Context Protocol (MCP), allowing AI assistants to interact with Dynatrace programmatically.

Setup

```
# Install and run via npx
npx -y @dynatrace-oss/dynatrace-mcp-server@2
```

Configuration (Claude Code / AI Assistants)

```
{
  "mcpServers": {
    "dynatrace": {
      "command": "npx",
      "args": ["-y", "@dynatrace-oss/dynatrace-mcp-server@2"],
      "env": {
        "DT_ENVIRONMENT": "https://{tenant}.apps.dynatrace.com",
        "DT_PLATFORM_TOKEN": "<platform-token>"
      }
    }
  }
}
```

The server reads `DT_ENVIRONMENT` (a platform `apps.dynatrace.com` URL, not a classic `live` URL) and authenticates with `DT_PLATFORM_TOKEN` or an OAuth client

(`OAUTH_CLIENT_ID` / `OAUTH_CLIENT_SECRET`), per the [dynatrace-mcp-server README \(Dynatrace GitHub\)](#). Pinning the major version (`@2`) keeps a breaking release from reaching your assistant unannounced.

MCP Server Capabilities

Capability	Description
DQL Queries	Execute DQL queries against Grail
Entity Discovery	Find and explore entities by name or type
Problem Analysis	Retrieve detected problems and events
Event Ingestion	Send custom events into Grail
Natural Language	AI assistants translate prompts to DQL

When to Use MCP vs SDKs

Scenario	Tool Choice
AI-assisted development	MCP Server
Custom application	TypeScript SDK (platform apps) or REST API
Automated pipeline	REST API or Monaco/Terraform in CI/CD
Interactive triage	MCP Server with Claude or Copilot
Batch processing	SDK

6. Next Steps

When to Use SDKs

Scenario	Tool Choice
Custom reporting tool	SDK
One-off query	Direct API or Notebooks

Scenario	Tool Choice
Config deployment	Monaco or Terraform
Integration app	SDK
Complex logic	TypeScript SDK, or Python against the REST API
AI-assisted access	MCP Server

Continue the Series

Next Notebook	Focus
AUTOM-07: CI/CD Integration	GitOps patterns and pipelines

Additional Resources

- [Dynatrace Developer Portal](#)
- [TypeScript SDK Documentation](#)
- [Settings 2.0 API \(DT docs\)](#)
- [API Reference](#)
- [MCP Server Documentation](#)

Summary

In this notebook, you learned:

- How to use the TypeScript SDK clients, and how to call the REST APIs from Python
- Common patterns for queries, settings, and entities
- Building custom CLI tools and reporting scripts
- Dynatrace MCP Server for AI-assisted programmatic access
- Best practices for SDK usage

Key Takeaway: SDKs provide the most flexibility for custom applications. Use them when you need programmatic access, complex logic, or integration with other

systems. For AI-assisted workflows, consider the MCP Server. For simple config management, Monaco or Terraform are often simpler.

Continue to AUTOM-07: CI/CD Integration to learn GitOps patterns.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.